

1. 왜 가능한가? (원리)

PDF 문서는 내부적으로 모든 텍스트, 선, 이미지의 **정확한 X, Y 좌표값**을 절대적으로 기록하고 있습니다. 웹 기술(HTML/CSS/Canvas) 역시 화면 상의 특정 픽셀 위치에 요소를 강제로 배치하는 기능(position: absolute)을 지원하므로, PDF의 좌표 데이터를 추출해 웹 화면에 그대로 1:1로 매핑하여 시각적인 복제가 가능합니다.

2. 어떻게 가능한가? (필요 기술)

이 작업을 구현하기 위해 데이터 추출부터 렌더링까지 다음과 같은 기술 스택이 복합적으로 필요합니다.

- **PDF 파싱(Parsing) 라이브러리**: PDF의 바이너리 구조를 읽고 텍스트, 폰트, 이미지, 벡터 그래픽의 좌표와 크기를 정밀하게 추출합니다. (예: PDF.js, PyMuPDF)
- **절대 좌표 기반 HTML/CSS 렌더링**: 추출된 요소를 픽셀 단위로 고정 배치합니다. 구조적인 태그(예: <table>) 대신, 개별 텍스트와 선을 <div> 태그로 캔버스에 흘뿌리는 방식을 사용합니다.
- **SVG 및 Canvas API**: 문서 내의 복잡한 테두리, 배경 색상, 또는 전자서명 완료 표시 등의 벡터/이미지 요소를 브라우저에 깨짐 없이 그리기 위해 사용됩니다.
- **통합 변환 엔진 활용**: 처음부터 모든 좌표 매핑을 직접 구현하기는 매우 어렵기 때문에, pdf2htmlEX와 같이 이미 스캐너 수준의 렌더링 정확도를 자랑하는 강력한 오픈소스 도구를 백엔드에 통합하여 사용하는 것이 가장 효율적입니다.

3. 어떤 한계가 있는가? (제약 사항)

시각적 완벽함을 추구할수록 웹 문서 본연의 장점을 잃게 되는 명확한 트레이드오프(Trade-off)가 발생합니다.

- **반응형 웹(Responsive Web) 적용 불가**: 모든 요소가 픽셀 단위로 화면에 고정되므로, 모바일 화면이나 브라우저 창 크기 변화에 맞춰 텍스트나 표의 레이아웃이 자연스럽게 재배치되지 않습니다.
- **의미론적(Semantic) 데이터 구조 상실**: 눈으로 보기에는 완벽한 표(Table)나 문단이지만, HTML 코드 상으로는 단순한 텍스트 조각들이 화면에 떠 있는 형태입니다. 따라서 프로그램이 표의 행과 열 데이터를 인식하거나 재가공하기 어렵습니다.
- **성능 및 파일 용량 문제**: 원본 문서와 동일한 폰트를 웹 폰트로 임베딩해야 하고, 수천 개의 DOM 요소(태그)가 생성될 수 있어 브라우저 렌더링 속도와 메모리 점유율이 현저히 나빠질 수 있습니다.
- **수정 및 편집의 어려움**: 글자 하나를 추가하거나 삭제하려 해도 연관된 다른 텍스트들의 레이아웃이 자동으로 밀려나지 않아 브라우저 상에서의 문서 편집 기능을 구현하기가 매우 까다롭습니다.

스캔본 수준의 시각적 완벽함이 절대적으로 중요한지, 아니면 디자인이 조금 투박해지더라도 텍스트 추출이나 모바일 대응 같은 데이터로서의 활용성이 더 중요한지에 따라 기술 접근 방식이 완전히 달라집니다. 구현하려는 웹 페이지의 최종 목적(단순 열람용 뷰어, 데이터 가공, 양식 편집 등)은 무엇에 가장 가까우신가요?